CODE APEX 3.0 | IM-02

QArena

Autonomous AI QA Engineer

A detailed project, system architecture, methodology, algorithms, technology stack, feature catalog, and hackathon presentation report.

EXPLORE	TEST	PROVE	REPORT
Map a web app's routes and accessible controls.	Create grounded user journeys and execute them in Chromium.	Replay candidate defects in clean contexts.	Deliver evidence, screenshots, traces, and reusable tests.

Project in one sentence

QArena accepts a web application URL and a plain-language description, explores the application in a real browser, proposes tests grounded in observed UI, judges behavior using deterministic evidence, replays suspected bugs, then produces a report and portable regression tests.

Prepared from the QArena repository and deployment documentation. Implementation status is described honestly: local behavior is available by default, while durable cloud execution requires Redis, an RQ worker, and S3-compatible storage.

1. Executive summary

QArena is a browser-based autonomous quality assurance system built for teams that need more than a screenshot or an AI-generated claim. It combines bounded website exploration, deterministic test planning, real-browser execution, a weighted multi-signal judge, replay verification, report generation, and export of passing tests.

The problem

Manual QA is slow to repeat; scripted tests require upfront authoring and often miss unexpected routes or edge cases. AI-only bug summaries can be plausible but unsupported. Long-running browser jobs are also a poor fit for short-lived serverless functions because browser state and in-memory events disappear with the process.

The QArena response

- Discover routes and controls before writing journeys, so generated actions can be checked against observed evidence.
- Keep the final bug gate deterministic: candidate findings are reproduced in fresh browser contexts before becoming confirmed findings.
- Store the chain of proof: journey, expected versus actual behavior, screenshots, logs, and Playwright trace.
- Convert successful journeys into ordinary pytest and Playwright tests that the team can keep and run outside QArena.
- Offer a key-free deterministic path; optional LLM assistance can help plan or phrase, but does not replace evidence checks.

Hackathon value proposition

QArena's strongest differentiator is an evidence-first verification loop. It treats generated bug ideas as suspects, not facts. This is a clear story for a jury: autonomous discovery proposes; structured contracts constrain; replay verifies; artifacts explain; exported tests preserve value.

Intended users

Small product teams, student project groups, QA engineers, developers validating a public or locally hosted app, and technical leads who need a reproducible bug dossier plus a starter regression suite.

2. System architecture

The codebase uses a Python-centered architecture. FastAPI and Jinja render the UI and serve APIs; Playwright runs the browser pipeline. In local mode, the web process starts an in-process background thread. In production mode, the web process submits work to Redis/RQ and a separate persistent worker executes Chromium. Redis is also the shared run/event store, while S3-compatible object storage holds durable artifacts.

Logical component diagram

USER BROWSER	FASTAPI WEB	REDIS + RQ	BROWSER WORKER	TARGET APP
Jinja pages, JS, SSE, report downloads	Input checks, project APIs, results, artifacts	Durable job queue, run status, event stream	Orchestrator, Chromium, stages 2-8	Public URL or locally launched app
<---- live progress and links ---->	---- enqueue job ---->	---- RQ job ---->	---- HTTP / browser ---->	

Artifact path: browser worker writes run evidence, results and PDF into a per-run artifact directory, then uploads files and a ZIP when QA_S3_BUCKET is configured. The web process can retrieve artifacts from object storage when it does not have the worker's local files.

Component responsibilities

Component	Responsibility	Key files
FastAPI application	Input validation, project/team APIs, launch/status, SSE, role filtering, report and artifact downloads.	app/main.py, api/index.py
Orchestrator	Coordinates run lifecycle, pipeline stages, persistence, event emission and final report generation.	core/orchestrator.py
Sandbox	Starts Chromium, isolates browser contexts, guards navigation, reuses auth state, applies timeouts.	core/sandbox.py
Explorer and AppMap	Crawls bounded same-origin GET paths, records accessible elements and route relationships.	core/explorer.py, core/contracts.py
Planner, Tester and Judge	Builds observed-UI tests, executes them and turns signal evidence into bug candidates.	core/planner.py, core/tester.py, core/judge.py
Verifier, Reporter, Exporter	Replays candidates, files reports, creates and self-verifies portable tests.	core/verifier.py, core/reporter.py, core/exporter.py
Coordination / Worker	Redis streams, RQ queue integration, worker entry point and object storage.	core/coordination.py, core/worker.py

Two operating modes

Mode	How it runs	Operational implication
Local	FastAPI launches a daemon pipeline thread; artifacts are local.	Simple setup; process restart loses active in-memory runs. Suitable for development and demos.
Production / Vercel front end	Web submits RQ work to Redis; persistent worker runs Playwright; S3-compatible storage keeps evidence.	Requires shared Redis and S3 settings on web and worker. Web functions should not run Chromium jobs themselves.

3. End-to-end methodology and workflow

An expedition is modeled as a pipeline rather than a single prompt. Its purpose is to keep discovery, test generation, execution, verification and reporting separate, with structured contracts between stages.

Stage	Operation	Output / control
1. Setup	Validate URL, description and optional credentials/team; enforce URL hygiene and run concurrency cap.	Run ID and accepted request.
2. Sandbox	Probe target reachability; optionally boot local app; seed/reset hook; Chromium, same-origin navigation guard and optional login-once storage state.	Isolated browser environment and run directory.
3. Explore	Bounded BFS-style traversal of same-origin GET links, selected guest routes, safe search probe and cart enrichment.	AppMap: routes, state signatures, roles, names, graph edges, observed status and catalog prices.
4. Plan	Optional LLM proposes cases; deterministic journey grammar is fallback. Validate every route/control against AppMap and reject invented actions.	Prioritized TestCase contracts with expected outcomes.
5. Execute and judge	Playwright performs actions; collect response failures, console/page errors, DOM findings, axe results, contract failures and screenshots.	Per-step StepEvidence and possible bug suspects.
6. Replay verification	Run each suspect up to three times in fresh contexts, re-evaluate using the deterministic judge and original oracle.	Confirm at 2/3 or greater; otherwise classify flaky/unproven.
7. Report and export	Deduplicate confirmed findings, assign deterministic severity, render Markdown/PDF, export passing journeys as pytest-playwright files and self-verify.	Bug reports, PDF with screenshots, evidence bundle and reusable tests.
8. Benchmark	When a planted-bug truth catalog is configured, match filed findings against expected bugs and calculate quality metrics.	Precision, recall, F1, reproducibility and missed-bug list.

Typical user workflow

- Create a project team and add contributors with frontend, backend, QA, DevOps or project-lead roles.
- Start an expedition with a target URL and a precise description of expected business rules. Add test credentials only if needed.
- Watch the live field log, page/route map, candidate list, replays and completion state.
- Review each confirmed finding, screenshot and trace; switch to a role-filtered view if working with a team.
- Download the PDF or complete evidence ZIP; use exported pytest tests in CI or local development.

Safe exploration policy

During mapping, QArena follows same-origin navigation links and uses GET navigation. It records buttons and forms but avoids clicking them during the crawl. It blocks configured destructive paths and off-domain navigation. Mutating interactions are reserved for explicit planned test journeys. If the target offers /__seed__, QArena uses it as a deterministic reset hook before probes or tests.

Authentication model

Optional credentials are used in one login flow, and successful browser storage state is reused for later contexts. This avoids repeating login for every route. Use a dedicated, non-production test account. Test actions may intentionally mutate the target, so run against authorized environments with safe test data. In production, restrict Redis and bucket credentials to the required service and prefix.

4. Territory mapping: what it is and why it matters

Territory mapping is QArena's structured model of the target website before the tests run. It is not a screenshot gallery. The AppMap records observed routes (states), HTTP statuses, representative URLs, stable element roles and accessible names, and edges describing how a user can move between pages.

How QArena builds the map

- Start from the target home page and a small set of guest-only login or signup routes discovered separately.
- Collect same-origin links and place unseen URLs into a breadth-first frontier, with a depth cap.
- Parse an accessibility snapshot to record links, buttons, textboxes, search boxes, headings and other selected roles.
- Canonicalize numeric route segments, such as /product/3 to /product/*, and omit query noise from route identity.
- Build a state signature from the route, HTTP status and normalized role/name list; SHA-256 helps collapse duplicate states and prevent loops.
- Run safe enrichment probes, such as search navigation and adding one cart item, so stateful pages expose controls that only appear after interaction.
- Save AppMap.json and a SQLite state/edge catalog; render an interactive network map with NetworkX and PyVis.

AppMap data model

Record	Fields / meaning	Value to later stages
State	state_id, canonical route, sample URL, hash and HTTP status.	Lets planner identify observed pages and route failures.
Element	Role, accessible name, enabled/required metadata.	Grounds click/fill/select steps in controls actually observed.
Edge	Source state, destination route and navigation label.	Shows discoverable user journeys and map relationships.
Price ledger	First observed product-name to integer-cent amount mapping.	Provides a deterministic comparison baseline for price/total checks.

Why a territory map is a USP

The map makes QArena's reasoning inspectable: a test can be connected to a page and element, while rejected actions can be explained as unobserved. It also creates a reusable artifact for demos and triage. This is especially useful when a natural-language model proposes an imaginary button or route: the map acts as a grounding boundary.

A clear jury explanation

Before QArena tests a site, it builds a territory map of the routes, page states and accessible controls it can observe. That map grounds the test plan, reveals uncovered areas and helps explain why an imagined action was rejected.

Limits of the map

- The crawler is intentionally bounded by state and action caps, depth and same-origin rules; it will not exhaustively cover a large application.
- Pages requiring complex state, client-only routing, multi-factor authentication or unusual flows may be missed.
- A route signature is a useful approximation, not a proof that two application states are semantically identical.
- Only controls represented in the accessible snapshot are available to the planner; poor accessibility can reduce test reach.

5. Important algorithms and design choices

5.1 Bounded breadth-first exploration

Explorer maintains a FIFO frontier of URLs and visits each eligible URL once. It follows same-origin GET links to a bounded depth and stops at MAX_CRAWL_ACTIONS=60 or MAX_STATES=22. This gives breadth-first coverage of nearby pages while protecting the target and limiting run time.

5.2 Canonical state signature

For each visited page, QArena creates a canonical route and hashes route + status + stable accessible role/name elements. Numeric IDs, UUID-like values, timestamps and other volatile tokens are normalized before hashing. The SHA-256 digest is shortened to 16 hexadecimal characters. The purpose is to identify repeat states and avoid infinite crawl loops; this is a crawler optimization, not a cryptographic security feature.

5.3 Grounded test planning and invented-step rejection

The optional planner can use an LLM to draft strict JSON TestCase records. If unavailable or invalid, a deterministic journey grammar proposes common flows such as signup, login, search, cart, promo and checkout. Before execution, every goto route and interactive role/name is checked against the AppMap. Any case with invented actions is rejected and shown in the report. Valid cases receive priority scores: base 50; edge-case +15; adversarial +25; checkout/pay +30; login/signup +10; promo +10; capped at 99. Up to 16 tests are selected.

5.4 Six-signal evidence judge

Tester captures structured evidence; Judge groups signals into candidate defect families. Each signal has a configurable weight. Scores sum distinct signal weights within an evidence group, capped at 1.0. A suspect survives if score is at least 0.50 or at least two different signals agree.

Signal	Evidence family	Weight	Examples
S1	Console errors	0.45	JavaScript errors and uncaught page exceptions.
S2	Network failures	0.50	Same-origin HTTP response with status 400 or above.
S3	Broken page items	0.30	Broken links or invalid/empty content patterns.
S4	Form checks	0.50	Invalid input accepted, missing rejection, money invariants.
S5	Accessibility	0.25 base; selected solo rules 0.55	Serious/critical axe violations and label/contrast checks.
S6	Intent checks	0.60	Deterministic business-rule mismatches, such as price or cart invariants.

Example: a network failure alone receives 0.50 and meets the threshold. Broken-page signal alone at 0.30 does not. Two independent signals that agree can qualify even if their combined score is below 0.50. Grouping prevents unrelated errors on one page from being merged into a single vague report.

5.5 Three-run replay verification

Each suspect's recorded steps and expected contract are replayed in fresh browser contexts up to three times. The verifier re-runs the deterministic judge and looks for the same evidence group, or a compatible group family with an overlapping signal. At least two matching replays are required to file the bug. Zero or one match becomes unproven/flaky. This reduces intermittent noise but cannot prove the root cause in source code.

5.6 Deduplication and severity

Reporter first groups by canonical route, normalized element and test ID, then coalesces related signal groups within a journey. It assigns a deterministic severity from business impact clues and signal families: Critical for money invariants or high-impact checkout failures; Major for form/flow correctness failures; Minor for many accessibility, console and low-impact route issues. The actual rules are heuristic and are not a substitute for human risk review.

5.7 Honest benchmark matching

If a planted-bug truth file is configured, the scorer matches each report to one unused truth item. Similarity adds 0.45 for overlapping signals, 0.25 for route agreement and up to 0.30 for keyword overlap. A match at 0.50 or above counts as a true positive. Precision = TP / filed reports; recall = TP / planted bugs; F1 = harmonic mean. Without ground truth, benchmark is unavailable; it should not be presented as a score.

5.8 Optional LLM with deterministic fallback

LiteLLM is optional. The default model string is configurable, and provider credentials activate an LLM planner and optional report title prose. When a key is absent or a call fails, deterministic planning/reporting continues. The final replay verification loop does not call an LLM. Token use is provider usage when returned; zero in key-free mode is expected and does not mean no computation occurred.

6. Technology stack in detail

Layer	Technology	Role in QArena
Language	Python 3.12+ / Python 3	Orchestration, crawl logic, contracts, judge, reports, scoring and worker.
Web framework	FastAPI + Uvicorn	Async HTTP API, run pages, health endpoint, SSE progress, downloads.
Templates and UI	Jinja2, HTML, CSS, browser JavaScript	Server-rendered workflow UI, live run log, map embedding, team/report controls.
Browser automation	Playwright Python sync API + Chromium	Real browser navigation, locator actions, network/console events, contexts, traces and screenshots.
Typed contracts	Pydantic v2	Structured AppMap, test steps, expected contracts, evidence, filed bugs and run result.
Crawl graph	NetworkX + PyVis	Represent route edges and render an interactive territory map.
Run metadata	SQLite + JSON files	Local AppMap state/edge persistence and per-run result/evidence artifacts.
Durable work queue	Redis + RQ	Separate browser worker jobs and shared run status in production mode.
Evidence storage	AWS S3 API via boto3	Durable screenshots, traces, reports, exports and ZIP evidence pack.
LLM adapter	LiteLLM (optional)	Provider-neutral optional structured planning and report prose; key-free fallback remains.
Test export	pytest + pytest-playwright	Portable passing journeys emitted and self-verified.
PDF output	ReportLab	Run summary, bug details, screenshots, advice and page footer.
Deployment	Docker, Render Blueprint, Vercel web adapter	Containerized web service and worker; Vercel front end requires external durable worker services.

Why this stack fits the project

- Python is a natural fit for the orchestration, structured evidence processing, reporting and test generation layers.
- Playwright uses a real Chromium browser, so findings are tied to actual rendered behavior and browser events.
- FastAPI provides simple typed endpoints and streaming for a lightweight hackathon UI.
- Redis/RQ separates long browser work from short web requests, addressing the serverless process-lifetime problem.
- S3-compatible object storage separates durable evidence from ephemeral web/worker filesystems.
- The LLM is optional, reducing dependence on API keys and preserving a predictable deterministic demo mode.

Should QArena recommend React, Next.js or Python?

Not from a browser bug alone. A wrong total, missing label, broken route or 500 response does not imply the target framework is wrong. A stack recommendation needs evidence about the target's source architecture, bottleneck and constraints. The current build does not automatically inspect an arbitrary target's source repository or identify its framework. Its guidance therefore recommends fixing the observed cause in the existing stack, and profiling before a migration.

For QArena itself, the current Python/FastAPI/Playwright stack is suitable for a browser QA pipeline. If run throughput becomes the bottleneck, first tune crawl/test/replay caps and worker concurrency, then measure CPU, memory, browser startup and queue wait. For user-facing target speed, add performance metrics such as navigation timing, Core Web Vitals or API latency before recommending framework changes.

7. Feature catalog and unique solution propositions

Feature	What it does	Why it matters
Natural-language app brief	User enters a URL and describes expected behavior in plain language.	Makes test goals accessible to non-specialists; business rules inform checks.
Territory map	Interactive route/state/edge visualization plus JSON and SQLite records.	Makes exploration visible and gives tests a grounding source.
Safe crawler	Same-origin, GET-only bounded exploration; blocks configured destructive paths.	Reduces risk while mapping an unfamiliar site.
Grounded test plan	Model-assisted or grammar-generated test cases, validated against observed routes/elements.	Reduces hallucinated buttons/routes and preserves explainable rejections.
Six-signal judge	Correlates console, network, broken items, forms, accessibility and intent evidence.	Finds different defect classes with a thresholded evidence score.
Pass-cubed verification	Replays candidates in fresh contexts three times; files only at 2/3+.	A strong jury-friendly anti-false-positive mechanism.
Evidence specimen cards	Expected/actual, severity, route, reproduction steps, screenshot and trace links.	Turns an alert into actionable engineering evidence.
Automated PDF	Creates a summary PDF with bug screenshots, likely-cause notes and method references.	Easy to share with judges, stakeholders and contributors.
Role-based team reports	Project members receive role assignments; reports can be filtered by technical role.	Lets frontend/backend/QA/DevOps/lead focus on relevant findings.
Shareable report link	Creates a share token for a role-filtered report view.	Makes handoffs easier without sending the full evidence archive.
Portable regression tests	Passing journeys export to pytest + Playwright and are self-run before delivery.	Avoids locking the team into QArena to retain the tests.
Planted-bug benchmark	Computes precision, recall, F1 and reproduction against supplied truth data.	Adds a measurable evaluation path for demos and research.
Optional durable worker	Redis queue, RQ browser worker, resumable events and S3-compatible evidence.	Addresses long-running jobs and ephemeral/serverless process limits.
Evidence-led fix suggestions	Heuristic likely cause, repair checklist, stack-neutral guidance and public vendor-method references.	Helps move from finding toward triage while labeling uncertainty.

The strongest USPs to emphasize to judges

- Proof before prose: a suspect does not become a bug report until replay verification passes.
- Grounded autonomy: route and element checks reject invented test actions and make limitations visible.
- Inspectable territory: the AppMap shows what the agent actually saw, rather than hiding the exploration process.
- Evidence continuity: findings can carry screenshots, browser traces, response data and exact reproduction steps.
- Portable value: successful journeys become plain tests the team owns and can run in CI.
- Key-free resilience: deterministic operation does not require an LLM API key; LLM use is optional assistance.
- Honest evaluation: planted-bug precision/recall/F1 can expose misses as well as successes.
- Deployment-aware design: long-running Playwright work runs on a persistent worker in production mode.

8. Implementation status, limits and responsible claims

This section separates current implementation from roadmap ideas so the presentation remains credible. A hackathon jury may ask whether every deployment mode or roadmap feature is fully working; explain the dependencies and boundaries directly.

Capability	Status in current repository	What to say
Redis/RQ queue and persistent worker	Implemented; requires QUEUE_MODE=redis and matching Redis configuration.	Production architecture exists; local mode intentionally runs in-process.
Shared status and resumable event stream	Redis event stream/SSE path implemented for configured production mode.	Events can be replayed from IDs; this depends on shared Redis.
Object storage	S3-compatible upload/retrieval conditional on QA_S3_BUCKET and credentials.	Evidence is durable only when the bucket is configured.
Screenshots, traces and log evidence	Implemented; PDF embeds the finding screenshot when the path exists.	A report depends on the run reaching report generation.
Replay verification	Three attempts and 2-of-3 threshold implemented.	Transient infrastructure exceptions are not yet given a separate failure class.
API + UI validation	Browser records same-origin HTTP failures; browser flows check UI contracts.	There is not yet a separate direct API contract test engine.
Changed-area regression selection	Not implemented.	The current repo-mode form does not perform a complete Git diff-based test selection.
Root cause and fix guidance	Evidence heuristics implemented; exact source-line proof is not.	Treat as likely cause and verify using source code/logs.
Target framework detection	Not implemented for arbitrary URL targets.	Do not claim automatic React/Next/Python stack detection.
Competitor method references	Static links to public vendor product documentation; no live competitor API integration.	These are public approaches, not claims about private internal technology.
Role access control	Role filtering and share links implemented; not a full identity/authorization system.	Roles organize views; do not describe as enterprise-grade access control.

Limitations to state proactively

- Coverage is bounded by crawl depth, state caps, timeouts, accessible semantics and test grammar.
- A run can stop due to target availability, browser installation, worker configuration, provider errors or reporting errors; zero filed bugs after a failed run is inconclusive.
- The judge uses heuristic weights and contracts. Human review remains important for severity, business impact and whether an observed behavior is truly a defect.
- The recommended root cause is a triage hypothesis unless a repository diff, stack trace or server log confirms it.
- Benchmark metrics are meaningful only with a credible, supplied ground-truth catalog and representative test runs.
- The code's current timeout/replay behavior can classify exceptions as non-matches; separating worker/infrastructure failure from application defects is a roadmap item.

9. Competitor methods and QArena's recommendation stance

The report can point engineers to publicly documented testing approaches from other products. It does not assert how a competitor privately implements its systems, and it does not claim that the competitor automatically fixes the target bug. Use the references as method benchmarks.

Public product / tool	Published method	How QArena can learn from it
Playwright Trace Viewer	Action timeline with DOM snapshots and action-filtered console/network logs.	QArena already stores traces; make its finding-to-trace handoff clear and use trace evidence to localize failures.
BrowserStack Accessibility Testing	Accessibility scans, rule-level reports and guidance; products also offer IDE-assisted workflows.	Attach rule IDs, impact and suggested remediation; add repeat scans across browsers/devices as a future feature.
Applitools Eyes	Visual checkpoint comparisons; documentation describes DOM/CSS root-cause details for visual differences.	Add baseline-based visual regression and viewport comparisons only after screenshot capture and diff review are implemented.

What QArena currently recommends

- For accessibility findings, review the cited axe rule, provide a persistent accessible name or correct contrast, then rescan.
- For HTTP failures, inspect the exact request URL/method/status and correlate it with server logs before deciding whether the route, response contract or upstream dependency is responsible.
- For JavaScript exceptions, use the trace's console stack and source map to locate the throwing component.
- For form findings, compare client-side and server-side validation, then test valid and invalid boundaries.
- Do not migrate from React to Next.js or to Python merely because a defect exists. Fix the demonstrated behavior in the current stack; profile before architecture changes.
- Use the longest measured QArena pipeline stage to optimize QArena itself. This is not yet a measurement of the target app's Core Web Vitals, server latency or API throughput.

Official public references

Playwright Trace Viewer documentation

BrowserStack Accessibility Testing

BrowserStack Accessibility DevTools

Applitools test and step editor / root cause analysis

Applitools visual testing overview

10. Deployment, performance and run artifacts

Local setup and run

The repository README provisions a Python virtual environment, installs requirements, installs Playwright Chromium, and starts Uvicorn on port 8088. The UI is available at http://localhost:8088. In local mode, no Redis or S3 is needed; runs and artifacts live on the local filesystem.

Production deployment

- Build the Docker image; it installs dependencies and Chromium with operating-system packages.
- Set QUEUE_MODE=redis and REDIS_URL on both web and worker services.
- Set QA_S3_BUCKET and AWS or S3-compatible endpoint credentials on both web and worker services.
- Deploy a web service and an RQ worker using the same queue name and artifact configuration.
- For Vercel, use the function only as the short-request web layer; the Playwright job belongs on a persistent worker.

Current performance controls

Control	Configured default	Purpose
Unique page states	22	Bounds map growth.
Crawl actions	60	Bounds navigation work.
Tests per run	16 maximum	Bounds planned test volume.
Steps per test	14 maximum	Bounds journey duration/complexity.
Replay checks	3 attempts; 2 required	Balances confidence and cost.
Navigation timeout	10 seconds	Stops indefinitely hanging navigation.
Run wall clock	480 seconds	Limits total pipeline time.
Browser concurrency	QA_MAX_ACTIVE_RUNS default 2	Protects the local service from too many concurrent runs.

Chromium consumes significant memory. Start with conservative worker concurrency, measure queue wait and stage duration, then scale workers horizontally. Reduce crawl/test/replay caps for a live demo only with the tradeoff stated. A slow QArena stage and a slow target app are separate performance problems.

11. Hackathon demo and jury presentation

Suggested 90-second demo

Time	Show	Say
0:00-0:10	Enter a target URL and behavior brief.	QArena starts from expected user outcomes, not pre-written test code.
0:10-0:25	Show territory map states, routes and controls.	The map grounds the planner and exposes what the agent actually observed.
0:25-0:40	Show a candidate and its evidence signals.	One anomaly is only a suspect; we preserve the evidence behind it.
0:40-0:55	Show three clean replays and 2/3 rule.	A finding must reproduce before it can be called confirmed.
0:55-1:10	Open bug card with screenshot, expected/actual, steps and trace.	An engineer can reproduce and investigate without trusting a generated paragraph.
1:10-1:20	Download PDF and show screenshots/recommendation.	The report is portable, with evidence and cautious remediation guidance.
1:20-1:30	Show exported pytest test and benchmark if truth data exists.	The team keeps regression assets and can measure accuracy against ground truth.

Jury pitch

'QArena is an autonomous web QA engineer that maps a website, plans tests from observed controls, and refuses to call a bug confirmed until it reproduces in clean browsers. Each finding comes with a route, expected versus actual behavior, reproduction steps, screenshot and Playwright trace. Passing journeys become ordinary tests the team owns. When deployed with Redis, a persistent worker and object storage, it can keep long-running browser jobs alive beyond a serverless request.'

Likely judge questions and strong answers

Question	Answer
How do you prevent AI hallucinations?	Planning output is structured and checked against the observed AppMap. Invented route/control steps are rejected. The bug gate is deterministic and replay-based.
How do you know a bug is real?	We require matching evidence on at least two of three clean-context replays. The report includes the observation and trace. Human review remains needed for intended product behavior and severity.
What happens without an API key?	The deterministic grammar planner and heuristic reporter continue. LLM token usage is zero because no provider call occurs.
Why not use a different framework?	The defect evidence does not imply a framework problem. QArena recommends fixing the measured cause in the current stack and profiling before migration.
Can it run on Vercel?	The web front end can, but long-lived Playwright work needs Redis/RQ and a separate persistent worker, plus durable storage for evidence.
What is not done yet?	Changed-area regression selection, dedicated API contract tests, target framework detection, and separate infrastructure-failure classification are roadmap items.

Final takeaway

QArena's differentiator is not simply 'AI tests a website.' It is an inspectable, evidence-carrying, replay-verified workflow that turns exploration into reproducible findings and reusable tests.

Appendix A. Terms and references

Term	Definition in QArena
AppMap / territory map	Structured routes, page states, controls and navigation edges discovered by Explorer.
ExpectedContract	Typed set of assertions such as required text, forbidden URL, reject behavior, badge value and non-negative money.
Signal	One evidence family used by Judge: console, network, broken item, form, accessibility or intent.
Suspect	A candidate bug meeting a signal threshold but not yet filed.
Confirmed finding	A suspect reproduced at least twice in three replay attempts.
Flaky / unproven	An observation that did not meet the replay gate or could not be checked.
Precision	Among filed bugs, the fraction matched to the supplied planted-bug truth set.
Recall	Among planted bugs in the supplied truth set, the fraction matched by filed reports.
Evidence pack	ZIP archive of the result, logs, map, screenshots, traces, reports and exported tests.

Repository references

- README.md and DEPLOYMENT.md: setup, local mode and production deployment.
- core/config.py: crawl, test, timeout, scoring and replay limits.
- core/contracts.py: typed AppMap, TestCase, evidence and result shapes.
- core/explorer.py, planner.py, tester.py, judge.py, verifier.py: main discovery and verification algorithms.
- core/reporter.py, exporter.py, pdf_report.py and scorer/evaluate_benchmark.py: outputs and evaluation.
- core/coordination.py, core/worker.py and render.yaml: queue, shared event stream, worker and cloud setup.
- app/main.py and frontend/: web UI, API and team/report workflow.

Important API surface and artifacts

API	Purpose
POST /api/runs	Validate and submit an expedition.
GET /runs/{id}	Render a run page and resume its live view.
GET /api/runs/{id}/events	Stream stage/progress events using SSE.
GET /api/runs/{id}/result	Retrieve the structured RunResult.
GET /api/runs/{id}/report/{role}	Return role-filtered bugs and recommendations.
GET /api/runs/{id}/report/{role}/download	Download role-filtered Markdown.
POST /api/runs/{id}/share	Create shareable role report token.
GET /artifacts/{id}/{path}	Serve local or object-store evidence artifacts.
GET /api/runs/{id}/zip	Download full evidence archive.

Per-run artifact set

results.json, run.log, AppMap.json, codemap.sqlite, map.html, step screenshots, Playwright trace ZIPs, bug_reports/*.md, bugs_filed.json, exported_tests/*.py, pytest_self_run.log and qarena_report.pdf. The ZIP evidence pack preserves these for offline review.

External product-method references

Playwright Trace Viewer

BrowserStack Accessibility Testing

BrowserStack Accessibility DevTools

Applitools root-cause analysis documentation

Report generated from the QArena project repository for hackathon planning and presentation. Claims about implementation status are intentionally scoped to the repository configuration and code reviewed.